Answers to Your Lecture Slip Questions

Lecture slip of October 20, 2026 (State-Space Search)

CSCI 235 – Software Design and Analysis II

Hunter College, Fall 2026

Questions from the Oct 20 slip

Contents (click an entry to jump)

1. Cheat sheet
2. Enqueue and dequeue
3. Slip question 1: search
4. Slip question 2: find
5. Slip question 3: double
6. Permutations and counting
7. Review and cheat sheet
8. More practice

Your questions → answers: cheat sheet (§1) enqueue/dequeue (§2) "multiply with `k+1`?" (§3) questions 2 and 3 (§4, §5) diagram of `find()` (§4) permutations, $\binom{n}{k}$, $n!$ (§6) review and cheat sheet (§7) more practice (§8)

Can We Use a Cheat Sheet During the Exam?

Only the provided cheat sheet is allowed

You may **not** bring your own notes or a self-made cheat sheet.

- The provided sheet is printed on the **last two pages** of your exam packet: one sheet, front and back.
- Everything else is closed book and closed notes.
- The version on the course schedule was **corrected on Oct 20**. Please **re-download** it, so you study from the same version you will get on the exam.

Summary of Enqueue and Dequeue

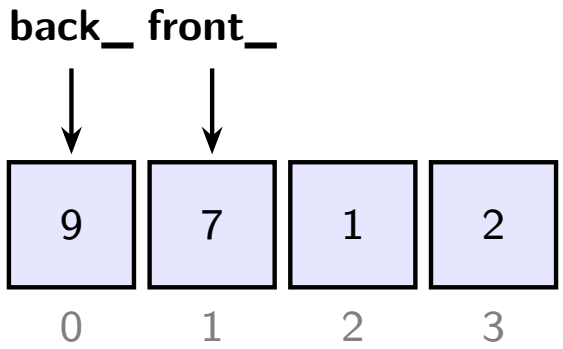
Circular queue, capacity C. `front_` = first item, `back_` = last item.
Empty: `front_ == -1 && back_ == -1`.

Operation	**If the queue is...**	**What happens**
`enqueue`	full	nothing
`enqueue`	empty	`front_ = back_ = 0`
`enqueue`	otherwise	`back_ = (back_ + 1) % C`
`dequeue`	empty	nothing
`dequeue`	one item (`front_ == back_`)	`front_ = back_ = -1`
`dequeue`	otherwise	`front_ = (front_ + 1) % C`

Size: if `front_ <= back_`: `back_ - front_ + 1`; if wrapped (`front_ > back_`): `C - front_ + back_ + 1`. Full when size $= C$.

Enqueue and Dequeue: A Worked Example ($C = 4$)

#	Operation	front_	back_	Contents (front first)	
1	enqueue(5)	0	0	5	
2	enqueue(7)	0	1	5, 7	
3	dequeue()	1	1	7	
4	enqueue(1)	1	2	7, 1	
5	enqueue(2)	1	3	7, 1, 2	
6	enqueue(9)	1	**0**	7, 1, 2, 9	(full)



Step 6: back_ wrapped around to index 0, so front_ $>$ back_ while the queue is full.

"Do We Multiply With `k+1`?" No.

`k` is a position

Which number of `v` comes next. `k + 1` means "move on to the next number".

`value` is the running result

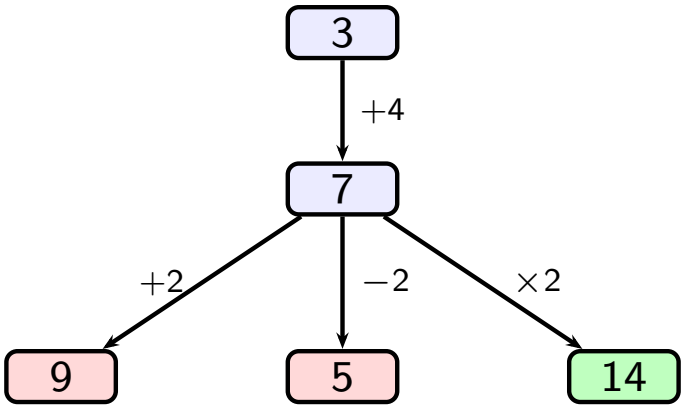
`value + x` is the new running result after adding the next number.

The four calls joined by `||` are four **separate branches**. Multiplication is one of the four choices; it does not come after addition.

```
return search(v, k + 1, value + x, target) // branch 1: add
    || search(v, k + 1, value - x, target) // branch 2: subtract
    || search(v, k + 1, value * x, target) // branch 3: multiply
    || (x != 0 && search(v, k + 1, value / x, target)); // 4: divide
```

Trace: `v = {3, 4, 2}`, Target 14

Step	Call	Next number `x`	What happens
1	`search(v, 1, 3, 14)`	$v[1] = 4$	try add first: $3 + 4 = 7$
2	`search(v, 2, 7, 14)`	$v[2] = 2$	try add first: $7 + 2 = 9$
3	`search(v, 3, 9, 14)`	—	$k = 3$ is a leaf; $9 \neq 14$, false
4	back in step 2	2	subtract: $7 - 2 = 5$, leaf, false
5	back in step 2	2	multiply: $7 \times 2 = 14$, leaf, **true**
6	`||` stops at the first true: no other branch is tried.		



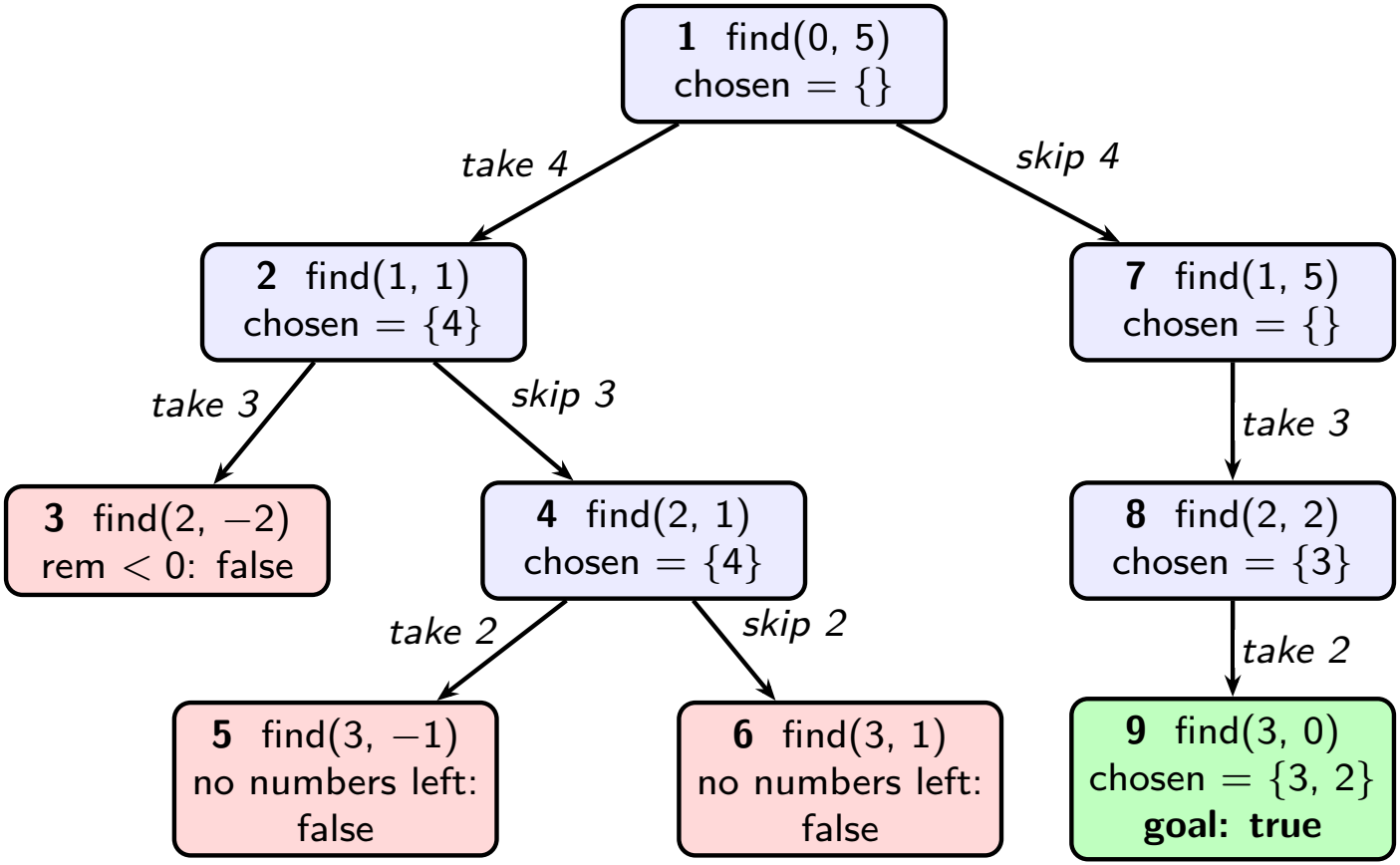
The search checks 3 leaves (9, 5, 14) out of $4 \times 4 = 16$ possible.

Question 2: `find`, Choose, Explore, Undo

```
bool find(const std::vector<int>& v, int k, int rem, std::vector<int
    >& chosen) {
  if (rem == 0) { return true; }                                  // goal
  if (k == static_cast<int>(v.size()) || rem < 0) { return false; } //
       dead end
  chosen.push_back(v[k]);                          // choose
  if (find(v, k + 1, rem - v[k], chosen)) { return true; } // explore
      : take
  chosen.pop_back();                               // undo
  return find(v, k + 1, rem, chosen);              // explore: skip
}
// v = {4, 3, 2}; first call: find(v, 0, 5, chosen)
```

At each number there are two choices: **take** it (subtract it from `rem`) or **skip** it. The function tries "take" first. If that fails, it **undoes** the choice and tries "skip".

Diagram of `find(v, 0, 5, chosen)`



Red: dead ends. Green: goal. Small numbers: order of the calls.

Step by Step, With the Undo

Call	Action	`chosen` after	Result
1 `(0,5)`	take 4	{4}	explore call 2
2 `(1,1)`	take 3	{4, 3}	explore call 3
3 `(2,−2)`	`rem < 0`: dead end	{4, 3}	false; **undo 3**: {4}
4 `(2,1)`	take 2	{4, 2}	explore call 5
5 `(3,−1)`	no numbers left	{4, 2}	false; **undo 2**: {4}
6 `(3,1)`	no numbers left	{4}	false; calls 4 and 2 fail
1	**undo 4**, skip 4	{}	explore call 7
7 `(1,5)`	take 3	{3}	explore call 8
8 `(2,2)`	take 2	{3, 2}	explore call 9
9 `(3,0)`	`rem == 0`: goal	{3, 2}	**true**, up the stack

Question 2: The Answers

(a) `chosen = {3, 2}`, and $3 + 2 = 5$.

(b) Without `chosen.pop_back();` the dead ends are never erased. The final `chosen` is `{4, 3, 2, 3, 2}`, whose sum is 14, not 5. The function still returns `true`, but the recorded answer is wrong: every later branch starts from a state that is not the one where we branched.

Rule to remember

Every `push_back` (choose) needs a matching `pop_back` (undo) on the failure path.

Question 3: The `double` Trap

```
double a = 0.5 + 0.25;
std::cout << (a == 0.75);              // line 1
double s = 0.0;
for (int i = 0; i < 10; i++) { s += 0.1; }
std::cout << (s == 1.0);               // line 2
```

(a) Line 1 prints `1`. Line 2 prints `0`.

(b) Binary storage: $0.5 = 2^{-1}$, $0.25 = 2^{-2}$, and 0.75 are sums of powers of two, so they are stored **exactly**. But 0.1 is not a finite binary fraction, just as $1/3$ is not a finite decimal (0.333 loses a little). Ten additions of `0.1` give about 0.9999999999999989.

(c) Compare with a tolerance: `std::abs(s - 1.0) < 1e-9`, the same test the search uses for its goal.

Permutations, $n!$, and $\binom{n}{k}$

Permutations are the topic of Class 15 (Oct 22). Not on the midterm.
Use 4 guests A, B, C, D.

Formula	Meaning	4 guests
$n! = n(n-1)\cdots 1$	arrangements of n *different* items (order matters)	4 chairs: $4 \times 3 \times 2 \times 1 = 24$
$\dfrac{n!}{(n-k)!}$	arrangements of k items chosen from n (order matters)	2 of 4 in 2 chairs: $4 \times 3 = 12$
$\binom{n}{k} = \dfrac{n!}{k!\,(n-k)!}$	ways to *choose* k of n (order does *not* matter)	choose 2 of 4: $12 \div 2 = 6$

The 12 ordered pairs count each pair twice (AB and BA), so $\binom{4}{2} = 12/2! = 6$. With repeated items, divide by each repeat's factorial: `"AAB"` has $3!/2! = 3$ arrangements.

Review of Today's Lesson (Oct 20)

1. **A search problem has states, moves, and a goal.** Maze: the cell you stand on. Make 24: the numbers left on the table.
2. **Choose, explore, undo.** Make a choice, explore what follows, then undo it so the next branch starts from the same state.
3. **Compare floating-point numbers with a tolerance,** never with ==.

State-space search is **not** on the midterm, so it is not on the cheat sheet.

What Is on the Cheat Sheet

1. Classes and polymorphism: `virtual`, `override`, slicing, copy constructor and copy assignment
2. Templates, MiniVector, and memory: Rule of Three, growth by doubling
3. Singly linked list: traversal, reversal, `popFront`
4. Stacks and queues: the circular queue rules
5. Recursion: base case, recursive step, call stack
6. P2: DFS and BFS on a maze
7. Big-O patterns

How to review with it. Redo the Oct 15 and Oct 20 slip questions with the sheet open, and notice which line you reach for. The sheet reminds you of the rules; you still need to practice applying them.

More Tracing Practice

Use `find` from question 2 with `v = {2, 3, 4}` and the first call `find(v, 0, 6, chosen)`.

(a) Draw the call tree like the one on the diagram slide. What is `chosen` when the call returns `true`?

(b) Delete `chosen.pop_back();`. What is `chosen` now? Is it a correct answer?

You can also redo question 2 of the Oct 15 slip (the function `g(n)`) and draw its call tree.

More Tracing Practice

Use `find` from question 2 with `v = {2, 3, 4}` and the first call `find(v, 0, 6, chosen)`.

(a) Draw the call tree like the one on the diagram slide. What is `chosen` when the call returns `true`?

(b) Delete `chosen.pop_back();`. What is `chosen` now? Is it a correct answer?

You can also redo question 2 of the Oct 15 slip (the function `g(n)`) and draw its call tree.

Check your work

(a) `chosen = {2, 4}`, since $2 + 4 = 6$: take 2, take 3, take 4 (dead end, undo), skip 4 (dead end), undo 3, skip 3, take 4, reach `rem == 0`.
(b) Without the undo, `chosen = {2, 3, 4, 4}`, sum 13, not 6. It still returns `true`.